

1. Data Augmentation

What is Data Augmentation?

Data Augmentation is a technique used to artificially increase the size of a training dataset by creating modified versions of existing images.

Instead of collecting more data, we transform existing images.

Why Use Data Augmentation?

- Reduces overfitting
- Improves model generalization
- Increases dataset diversity
- Useful when dataset is small

Common Augmentation Techniques

1. Rotation

Rotate image by a certain angle.

```
rotation_range=30
```

2. Horizontal Flip

```
horizontal_flip=True
```

3. Zoom

```
zoom_range=0.2
```

4. Width Shift

```
width_shift_range=0.2
```

5. Height Shift

```
height_shift_range=0.2
```

6. Shear Transformation

```
shear_range=0.2
```

Practical Example

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

train_generator = datagen.flow_from_directory(
    "dataset/train",
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical'
)
```

Modern TensorFlow Augmentation Layer

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.2),
    tf.keras.layers.RandomZoom(0.2),
])
```

2. Transfer Learning

What is Transfer Learning?

Transfer Learning means using a pre-trained model trained on a large dataset and adapting it for a new task.

Example:

- Model trained on ImageNet (14 million images)
 - Reuse knowledge for your own dataset
-

Why Use Transfer Learning?

- Faster training

- Better accuracy
- Requires less data
- Less computational cost

Popular Pretrained Models

Model	Size
VGG16	Large
VGG19	Large
ResNet50	Medium
MobileNetV2	Small
EfficientNet	Best Accuracy

Workflow

```
Pretrained Model
    ↓
Remove Output Layer
    ↓
Add New Output Layer
    ↓
Train on New Dataset
```

Practical Example

```
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2

base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)

base_model.trainable = False

model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(2, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

3. Fine Tuning

What is Fine Tuning?

Fine Tuning means unfreezing some layers of a pretrained model and retraining them on the new dataset.

Transfer Learning:

- Freeze pretrained layers

Fine Tuning:

- Unfreeze some pretrained layers
-

Why Fine Tune?

Transfer learning may not fully adapt to your data.

Fine tuning allows the model to learn task-specific features.

Workflow

Step 1:

Train new classifier layers

Step 2:

Unfreeze top layers

Step 3:

Train entire model with low learning rate

Practical Example

```
base_model.trainable = True

for layer in base_model.layers[:-20]:
    layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(0.00001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
```

```
    train_generator,  
    epochs=10  
)
```

Transfer Learning vs Fine Tuning

Transfer Learning	Fine Tuning
Freeze model	Unfreeze layers
Fast	Slower
Less data needed	More data needed
Lower computation	Higher computation

4. RNN (Recurrent Neural Network)

What is RNN?

RNN is a neural network designed for Sequential Data.

Examples:

- Text
- Speech
- Time Series
- Stock Prediction

Unlike ANN, RNN remembers previous information using Hidden States.

RNN Architecture

```
Input  
↓  
Hidden State  
↓  
Output  
  
Previous Hidden State  
↓  
Current Hidden State
```

Formula

$$h_t = f(W_h h h_{t-1} + W_x h x_t + b)$$

Where:

- h_t = current hidden state
 - h_{t-1} = previous hidden state
 - x_t = current input
-

Problem of RNN

Vanishing Gradient

When sequence becomes long:

Very Long Sentence

↓

Information Lost

↓

Poor Prediction

This problem led to LSTM and GRU.

RNN Example

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential([
    SimpleRNN(50, input_shape=(10,1)),
    Dense(1)
])

model.compile(
    optimizer='adam',
    loss='mse'
)

model.summary()
```

5. LSTM (Long Short-Term Memory)

What is LSTM?

LSTM is an improved version of RNN.

It can remember information for a long period.

Used in:

- Chatbots
 - Language Translation
 - Speech Recognition
 - Time Series Forecasting
-

Components of LSTM

1. Forget Gate

Decides what information to forget.

2. Input Gate

Decides what information to store.

3. Output Gate

Decides what information to output.

LSTM Architecture

```
Forget Gate
    ↓
Input Gate
    ↓
Cell State
    ↓
Output Gate
```

LSTM Example

Stock Price Prediction Style Data

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

X = np.random.rand(1000,10,1)
y = np.random.rand(1000)
```

```
model = Sequential([
    LSTM(64, input_shape=(10,1)),
    Dense(1)
])

model.compile(
    optimizer='adam',
    loss='mse'
)

model.fit(
    X,
    Y,
    epochs=10,
    batch_size=32
)
```

Multiple LSTM Layers

```
model = Sequential([
    LSTM(128, return_sequences=True,
        input_shape=(10,1)),
    LSTM(64),
    Dense(1)
])
```

Advantages of LSTM

- Handles long sequences
 - Better memory
 - Solves vanishing gradient problem
 - High accuracy
-

6. GRU (Gated Recurrent Unit)

What is GRU?

GRU is a simplified version of LSTM.

It requires fewer parameters and trains faster.

Gates in GRU

Update Gate

Controls how much past information to keep.

Reset Gate

Controls how much previous information to forget.

GRU Architecture

Input
↓
Update Gate
↓
Reset Gate
↓
Output

GRU Example

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GRU, Dense

X = np.random.rand(1000,10,1)
y = np.random.rand(1000)

model = Sequential([
    GRU(64, input_shape=(10,1)),
    Dense(1)
])

model.compile(
    optimizer='adam',
    loss='mse'
)

model.fit(
    X,
    y,
    epochs=10,
    batch_size=32
)
```

LSTM vs GRU

Feature	LSTM	GRU
Gates	3	2
Memory Cell	Yes	No

Feature	LSTM	GRU
Parameters	More	Less
Training Speed	Slow	Fast
Accuracy	Slightly Better	Slightly Lower
Complexity	High	Low

Complete Comparison

	ANN	CNN	RNN	LSTM	GRU
Tabular Data	✓	✗	✗	✗	✗
Image Data	✗	✓	✗	✗	✗
Sequential Data	✗	✗	✓	✓	✓
Long Sequence	✗	✗	Poor	Excellent	Excellent
Training Speed	Fast	Medium	Medium	Slow	Slow

Questions

1. What is Data Augmentation?
2. Why is Transfer Learning useful?
3. Difference between Transfer Learning and Fine Tuning?
4. What is a pretrained model?
5. Explain RNN architecture.
6. What is Vanishing Gradient Problem?
7. Why was LSTM introduced?
8. Explain Forget Gate in LSTM.
9. Difference between LSTM and GRU?
10. When should we use GRU over LSTM?
11. What is Hidden State in RNN?
12. What is Cell State in LSTM?
13. Why is MobileNetV2 popular for Transfer Learning?